

Girdi İşlemede Temsil Boşluğu

Ömer Faruk Yavuz

August 2026

Özet

Bir girdi üzerinde yapılan doğrulama ile o girdinin sonradan kullanılan hâli aynı değer olmadığında, aradaki fark bir güvenlik açığı üretir. Bu durum literatürde adlandırılmış bir zafiyet sınıfıdır: CWE-180, *Validate Before Canonicalize*. Sondaki boşlukların kırılması bu sınıfın en zararsız örneğidir; aynı şekil, dizin geçişi (*path traversal*), Unicode normalizasyonu ve imza doğrulaması gibi çok daha ağır sonuçlu bağlamlarda tekrarlanır. Sorunun kaynağı programcının dikkatsizliği değil, işlem sırasının kendisidir: doğrulama, verinin nihai temsili üzerinde yapılmadığı sürece hangi değeri onayladığını bilemez. Çözümün kontrol disiplini ile aranması yetersiz kalır, çünkü `bool` döndüren bir doğrulama fonksiyonu öğrendiği bilgiyi çağırana taşımaz; kalıcı çözüm, doğrulamayı bir tip dönüşümüne çevirmektir. Bu yaklaşımın literatürdeki adı *parse, don't validate*'tir (King, 2019). Kırpmaya ve hoşgörölü ayrıştırma alışkanlığının gerekçesi olarak gösterilen sağlamlık ilkesi (*robustness principle*) ise bugün İnternet Mimari Kurulu tarafından bizzat sorunun kaynaklarından biri sayılmaktadır (RFC 9413, 2023).

Giriş

Yazılımda bir girdi genellikle iki ayrı işlemden geçer. Birincisi **doğrulama** (*validation*): girdinin kabul edilebilir olup olmadığına karar veren, tipik olarak doğru ya da yanlış üreten işlem. İkincisi **kanonikleştirme** (*canonicalization*): aynı anlama gelen farklı gösterimleri tek bir standart biçime indirgeyen işlem. Boşluk kırpma, büyük-küçük harf katlaması, URL kod çözümü, dosya yolu çözümlemesi ve Unicode normalizasyonu bu işlemin örnekleridir.

Bu iki işlemin sırası, çoğu geliştirici tarafından bir tercih meselesi gibi görülür. Değildir. Doğrulama kanonikleştirmeden önce çalıştığında, doğrulanan değer ile kullanılan değer birbirinden ayrışır ve aralarında bir fark açılır. Bu fark saldırganın çalışma alanıdır: doğrulamadan geçen ama kullanım anında başka bir şeye dönüşen bir girdi üretmek yeterlidir.

Bu durum için gündelik konuşmada bir ad uydurmaya gerek yoktur; alanın kendi adlandırması mevcuttur. Aşağıda önce olgunun mekanizması, ardından yakın akrabalarından ayrımı, sonra kalıcı çözümü ve son olarak bu tür bilgilerin toplandığı taksonomi altyapısı ele alınmaktadır. Notun kapsamı girdi işleme sırasıyla sınırlıdır; kimlik doğrulama, yetkilendirme ve şifreleme gibi diğer güvenlik alanları yalnızca örnek düzeyinde geçmektedir.

Doğrulama ile Kanonikleştirme Arasındaki Boşluk

Mekanizma

Ortak Zayıflık Sıralaması (*Common Weakness Enumeration*, CWE) kataloğunda bu zafiyet CWE-180 numarasıyla kayıtlıdır. Tanım kısadır: ürün, girdiyi kanonikleştirilmeden önce doğrular ve bu nedenle kanonikleştirme sonrasında geçersiz hâle gelen veriyi tespit edemez (MITRE, CWE-180). Kardeş kayıt CWE-179 (*Incorrect Behavior Order: Early Validation*) aynı ailenin daha genel hâlidir.

Doğru sıra üç adımlıdır ve bozulduğu anda boşluk açılır:

1. Kanonikleştir.
2. Doğrula.
3. Kullan.

Kanonik örnek: dizin geçişi

MITRE'nin kendi sayfasında verilen örnek, sorunun neden bir dikkat meselesi olmadığını gösterir. Bir kod, verilen yolun `/safe_dir/` ile başlayıp başlamadığını denetler ve ardından yolun kanonik hâlini döndürür. Saldırgan `/safe_dir/./` girdisini gönderdiğinde denetim başarıyla geçer, çünkü dizge gerçekten de beklenen önekle başlamaktadır. Ancak kanonikleştirme işlemi çift noktayı üst dizine geçiş olarak yorumlar ve yol `/` hâline gelir (MITRE, CWE-180). Doğrulan yol ile açılan yol artık aynı yol değildir.

Boşluk kırpma hâli

Aynı şeklin en sık karşılaşılan ve en az fark edilen hâli, sondaki boşluklardır. Doğrulama kırpılmış dizge üzerinde, kayıt ya da karşılaştırma ham dizge üzerinde yapıldığında iki farklı değer ortaya çıkar:

```
if isValid(strings.TrimSpace(username)) {
    createUser(username)          // ham hâl kaydediliyor
}
```

Burada `"admin "` girdisi `"admin"` olarak doğrulanır, fakat `"admin "` olarak saklanır. Sistemin geri kalanı bu iki dizgeyi eşit sayıyorsa çakışma, saymıyorsa hayalet kayıt üretilir. Her iki sonuç da istenmez.

Kodlama ve normalizasyon hâlleri

Fark yalnızca boşluktan doğmaz. Karakter kodlaması söz konusu olduğunda kanonikleştirmenin kendisi tehlikeli değerler üretebilir. Unicode uyumluluk normalizasyonu (NFKC ve NFKD kipleri) bazı karakterleri düşürerek zararsız görünen bir dizgeyi zararlı hâle getirebilir; bu ayrı bir kayıt olarak CWE-182 (*Collapse of Data into Unsafe Value*) altında sınıflandırılmıştır. Benzer biçimde aynı girdinin

iki kez kod çözümüne tabi tutulması CWE-174 kapsamına girer. Açık Kaynak Güvenliği Vakfı'nın güvenli kodlama kılavuzu, izin listesi tabanlı doğrulamanın bu yollarla aşılabileceğini belirterek normalizasyonun doğrulamadan önce yapılmasını şart koşar (OpenSSF, Secure Coding Guide).

Teorik değil, güncel

Bu sınıfın soyut bir kategori olmadığına kanıtı üretimden gelir. 2026 yılında kayda alınan CVE-2026-42462, Fedify kütüphanesinde tam olarak bu şekli taşır: JSON-LD belgelerinde imza doğrulaması kanonikleştirmeden önce yapıldığı için saldırgan, imzayı geçersiz kılmadan belgenin yapısını değiştirebilmekte ve kütüphanenin belgeyi farklı yorumlamasını sağlayabilmektedir. Zafiyetin CVSS puanı 7.0 olarak belirlenmiştir.

Zaman Ekseni ile Temsil Ekseni Ayrımı

Bu zafiyet sıklıkla TOCTOU ile karıştırılır. TOCTOU, *time-of-check to time-of-use* ifadesinin kısaltmasıdır ve CWE-367 numarasıyla kayıtlıdır. Klasik örneği dosya sistemlerinde geçer: bir dosyanın erişim izni denetlenir, ardından dosya açılır, fakat iki işlem arasında geçen sürede izin değişmiştir.

İki durumun ortak yanı, sloganlaştırılabilecek tek bir cümlede toplanır: *doğrulanmış şey, kullanılan şey değildir*. Ayrıldıkları eksen ise farklıdır.

	TOCTOU (CWE-367)	Validate/Canonicalize (CWE-180)
Eksen	Zaman	Temsil
Değer	Aynı değer, iki an	Aynı an, iki gösterim
Değişen	Dış dünya	Verinin biçimi
Tipik alan	Dosya sistemi, yarış koşulu	Girdi işleme, ayrıştırma

TOCTOU'da aradaki fark dış bir olaydan doğar; kimse kodu yanlış yazmasa bile başka bir süreç araya girebilir. CWE-180'de ise dışarıda hiçbir şey değişmez. Aynı milisaniyede, aynı bellek içeriği üzerinde, iki farklı yorum uygulanır. Bu ayrım pratikte önemlidir çünkü çözümleri de farklıdır: TOCTOU kilitleme ve atomik işlemlerle, CWE-180 işlem sırası ve tip disipliniyle giderilir.

Sabit Genişlikli Alanlar ve Veritabanı Katmanı

Boşluk kaynaklı uyumsuzluğun en eski hâli, sabit genişlikli veri formatlarında görülür. DBF dosyaları, COBOL kayıt yapıları ve ana bilgisayar (*mainframe*) çıktılarında alanlar sabit uzunluktadır; kısa değerler boşlukla doldurulur. Bu doldurma işlemine **boşluk dolgusu** (*space padding*) denir. Değeri okuyan taraf dolguyu kırpar, kırpmayan taraf ham hâliyle kullanır ve iki taraf aynı kaydı farklı görür.

Aynı ayırım ilişkisel veritabanlarında tip düzeyinde kurumsallaşmıştır. SQL standardında `CHAR(n)` sabit genişliklidir ve karşılaştırma sırasında sondaki boşluklar yok sayılır; `VARCHAR` ise değişken genişliklidir ve boşlukları anlamlı sayar. Sonuç, aynı iki değerın sütun tipine göre eşit ya da eşit olmamasıdır. PostgreSQL üzerinde bu davranış doğrudan gözlenebilir:

```
SELECT 'ali '::char(10) = 'ali';      -- true
SELECT 'ali '::varchar = 'ali';      -- false
```

Bu, doğrulama-kanonikleştirme boşluğunun veri katmanındaki karşılığıdır. Uygulama katmanında yapılan kırpma ile veritabanı katmanının eşitlik tanımı uyummadığında, tekillik kısıtları (*unique constraint*) beklenmedik biçimde davranır: uygulama iki değeri farklı sayarken veritabanı aynı sayabilir ya da tersi olabilir.

Sağlamlık İlkesi ve Bugünkü Eleştirisi

Girdiyi kırpm, düzeltme ve hoşgörüyle kabul etme alışkanlığının gerekçesi olarak genellikle **sağlamlık ilkesi** (*robustness principle*) gösterilir. Jon Postel tarafından formüle edilen ilke, “gönderirken tutucu ol, alırken hoşgörülü ol” biçiminde özetlenir ve İnternet protokollerinin tasarımını onlarca yıl boyunca şekillendirmiştir (RFC 761; RFC 1122; RFC 1958).

Ancak bu gerekçelendirme bugün geçerliliğini yitirmiştir. İnternet Mimari Kurulu’nun (IAB) yayımladığı RFC 9413, ilkenin farklı biçimlerde yorumlandığını, bazı yorumlarının İnternet’in sağlığına katkı sağlarken diğerlerinin zaman içinde birlikte çalışabilirliği olumsuz etkilediğini tespit eder ve ilkeyi uygulamak yerine protokollerin etkin biçimde bakımını yapmayı önerir. Belgenin taslak hâlinin başlığı ilkenin nasıl değerlendirildiğini açıkça gösterir: *The Harmful Consequences of the Robustness Principle* (Thomson, 2018).

Eleştirinin mantığı doğrudan bu notun konusuna bağlanır. “Alırken hoşgörülü ol” talimatı, aynı baytların birden fazla geçerli yorumunun bulunmasına izin verir. Uygulamalar birbirinin hatalarına uyum sağlamaya başladığında, hangi yorumun doğru olduğu belirsizleşir. Doğrulama-kanonikleştirme boşluğu tam olarak bu belirsizlik alanında yaşar. Dolayısıyla sağlamlık ilkesi, kırpmanın gerekçesi değil, kırpm ile kullanım arasındaki uyumsuzluğun üreticisidir.

Kalıcı Çözüm: Doğrulamak Yerine Ayrıştırmak

İşlem sırasını düzeltmek gerekli fakat yeterli değildir, çünkü doğru sıra her çağrı noktasında yeniden hatırlanmak zorundadır. Unutulduğunda derleyici uyarır.

Alexis King'in 2019 tarihli yazısında formüle ettiği *parse, don't validate* ilkesi bu yapısal eksikliği hedefler. Tespit şudur: doğrulama fonksiyonu `bool` döndürdüğünde, öğrendiği bilgi çağırana tarafa aktarılmaz. Denetim başarılı olsa bile elde kalan hâlâ ham değerdir ve kullanılan da odur. Üstelik dönüş değeri kullanılmadığı için çağrı tümüyle atlanabilir; kod yine derlenir (King, 2019).

Ayrıştırma (*parsing*) bu sorunu tip düzeyinde çözer. Ayrıştırıcı, geniş bir tipten dar bir tipe dönüşüm yapar ve başarısızlığı dönüş değerinde taşır. Doğrulama sonucu artık atılmaz; tipin içine kodlanmıştır. King'in kendi ifadesiyle amaç, geçersiz durumları temsil edilemez kılan bir veri yapısı kullanmaktır.

Go dilinde bu, dar tipin dışı kapatılması ve yalnızca kurucu fonksiyon üzerinden üretilmesiyle yaklaşık olarak sağlanır:

```
package user

type Username struct {
    v string          // dışı kapalı; paket dışından atanamaz
}

func (u Username) String() string { return u.v }

func ParseUsername(raw string) (Username, error) {
    canonical := strings.TrimSpace(raw)
    canonical = norm.NFC.String(canonical)

    if len(canonical) < 3 {
        return Username{}, ErrTooShort
    }
    return Username{v: canonical}, nil
}
```

Bu düzenlemede kanonikleştirme ile doğrulama tek bir yerde ve doğru sırada gerçekleşir. Daha önemlisi, sistemin geri kalanı `Username` tipini gördüğü anda değerin kanonik ve geçerli olduğunu bilir; ham dizgeyi kullanmak paket dışından mümkün değildir. Boşluk kapanmıştır çünkü ham değer erişilebilir bırakılmamıştır.

Yaklaşımın sınırı da belirtilmelidir. Tip sistemi zayıf olan ya da dinamik tipli dillerde bu güvence derleme zamanında sağlanamaz; disiplin çalışma zamanı kontrollerine ve gözden geçirmeye kalır. Ayrıca her alan için ayrı tip tanımlamak küçük projelerde orantısız bir yük yaratabilir. Uygulamanın maliyeti, verinin güven sınırından (*trust boundary*) geçip geçmediğine bakılarak değerlendirilmelidir.

Zafiyet Taksonomileri ve CWE'nin Yeri

Yukarıda kullanılan numaralar rastgele değil, ortak bir kataloğun parçalarıdır. Bu altyapının haritası, bir bulgunun nereye kaydedileceğini ve nasıl önceliklendirileceğini belirlediği için pratik değer taşır.

CVE ile CWE ayrımı

CVE (*Common Vulnerabilities and Exposures*) somut bir olayı kaydeder: belirli bir üründe, belirli bir sürümde bulunan tek bir zafiyet. **CWE** ise sınıfı kaydeder: hangi üründe görülürse görülsün aynı hata tipini. İlişki örnekleyicilik ilişkisidir; her CVE bir veya birden fazla CWE'nin somut örneğidir. Yukarıda anılan CVE-2026-42462 kaydının CWE-180 ile etiketlenmesi bu ilişkinin doğrudan örneğidir.

CWE'nin varlık nedeni terminoloji birliğidir. Aynı hata farklı araçlarda farklı adlarla raporlandığı sürece bulgular karşılaştırılmaz. Kataloğun kayıtları ayrıca hiyerarşiktir; genelden özele inen ebeveyn-çocuk ilişkileriyle bağlanır. Katalogda dokuz yüzü aşkın kayıt bulunur, fakat pratikte en çok başvuru alan alt küme, gerçek CVE verisinden türetilen ve yıllık güncellenen *CWE Top 25* listesidir.

Çevredeki diğer yapılar

- **CAPEC** — Saldırı desenleri kataloğu. CWE kayıtlarındaki ilgili saldırı deseni bağlantıları buraya çıkar; “bu zayıflık pratikte nasıl sömürülür” sorusunu yanıtlar.
- **ATT&CK** — Saldırganın kurum içindeki davranışlarını sınıflandıran çerçeve. Kod düzeyinde değil operasyon düzeyinde çalışır.
- **CVSS** — Zafiyetin teorik şiddetini 0–10 aralığında puanlayan sistem. Tek başına önceliklendirme için yetersiz kalır.
- **EPSS** — Bir zafiyetin yakın vadede gerçekten sömürülme olasılığını tahmin eden model. CVSS'in eksiğini kapatmak üzere geliştirilmiştir.
- **KEV** — ABD Siber Güvenlik ve Altyapı Güvenliği Ajansı'nın (CISA) yayımladığı, hâlihazırda aktif olarak sömürülen zafiyetler listesi.
- **OSV, SLSA, SBOM** — Tedarik zinciri tarafı: paket zafiyet veritabanı, derleme hattı bütünlüğü çerçevesi ve yazılım bileşen listesi formatları.

OWASP ile ilişkisi

Açık Dünya Uygulama Güvenliği Projesi'nin (OWASP) yayımladığı Top 10, bağımsız bir taksonomi değil, CWE kayıtlarının risk temelinde kümelenmiş hâlidir. Listenin 2025 sürümü, dört yıllık aranın ardından Kasım 2025'te Washington'daki Global AppSec konferansında duyurulmuş, nihai hâli Ocak 2026'da yayımlanmıştır. Bu sürümde iki yeni kategori eklenmiştir: A03 Yazılım Tedarik

Zinciri Hataları ve A10 İstisnai Durumların Yanlış Ele Alınması. Sunucu Tarafı İstek Sahteciliği (SSRF) ise ayrı bir kategori olmaktan çıkarılıp Bozuk Erişim Kontrolü altına alınmıştır. Listenin dayandığı veri kümesi 2021'deki yaklaşık dört yüz CWE'den 589 CWE'ye genişlemiştir.

Top 10 bir farkındalık belgesidir; doğrulanabilir madde listesi arayanlar için OWASP'ın *ASVS* standardı, doğrudan uygulanabilir teknik rehber arayanlar için *Cheat Sheet Series* daha uygundur.

Sonuç

Doğrulama ile kanonikleştirme arasındaki sıra, üslup tercihi değil güvenlik sınırıdır. Bir girdi üzerinde yapılan denetimin geçerliliği, denetlenen değerin nihai temsil olmasına bağlıdır; aksi hâlde denetim, kullanılmayacak bir değeri onaylamış olur. Sondaki boşluk ile dizin geçişi arasındaki fark yalnızca sonucun ağırlığındadır, mekanizma aynıdır.

Bundan çıkan pratik ayrım şudur: sıra düzeltmek hatayı giderir, tip düzeltmek hatanın sınıfını ortadan kaldırır. İşlem sırasını doğru kurmak her çağrı noktasında yeniden hatırlanmayı gerektirdiği için kırılgandır; ham değeri erişilemez kılmak ise unutmayı imkânsızlaştırır. Güven sınırından geçen veriler için ikinci yaklaşımın maliyeti karşılanmaya değerdir.

Çerçevenin bulanıklaştığı yer, kanonik biçimin kendisinin tartışmalı olduğu durumlardır. Unicode normalizasyonunda hangi kipin seçileceği, uluslararasılaştırılmış alan adlarında hangi gösterimin esas alınacağı ya da JSON-LD gibi yapısal formatlarda kanonik serileştirmenin ne olduğu, teknik olduğu kadar tanım gerektiren sorulardır. Kanonik biçim üzerinde anlaşma sağlanamadığı sürece sıra doğru kurulsa bile boşluk tümüyle kapanmaz.

Notun cevaplamadığı ancak kurduğu soru, bu kararın hangi katmanda verileceğidir. Kanonikleştirme uygulama sınırında, veri erişim katmanında ve veritabanı motorunda ayrı ayrı yapılabilir; üç katmanın tanımları uyumadığında boşluk ortadan kalkmaz, yalnızca yer değiştirir. Katmanlar arası kanonik biçim mutabakatı, tekil bir kod düzeltmesinden çok mimari bir karardır.

Son olarak terminolojiye ilişkin bir uyarı yerinde olur. Bu olgu için sohbet içinde üretilen geçici adlandırmalar iletişimi kolaylaştırırsa da kalıcı kayıtlarda ve ekip içi yazışmalarda kullanılmamalıdır. Kataloğun kendi adı — CWE-180 ya da *validate before canonicalize* — karşı tarafın doğrulayabileceği tek referanstır.

Kaynaklar

- King, A. (2019). Parse, don't validate. *Alexis King's Blog*. <https://lexilambda.github.io/blog/2019/11/05/parse-don-t-validate/>
- MITRE. CWE-180: Incorrect Behavior Order: Validate Before Canonicalize. <https://cwe.mitre.org/data/definitions/180.html>
- MITRE. CWE-179: Incorrect Behavior Order: Early Validation. <https://cwe.mitre.org/data/definitions/179.html>
- MITRE. CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition. <https://cwe.mitre.org/data/definitions/367.html>
- MITRE. CWE-182: Collapse of Data into Unsafe Value. <https://cwe.mitre.org/data/definitions/182.html>
- OpenSSF Best Practices Working Group. Secure Coding Guide for Python: CWE-180. <https://best.openssf.org/Secure-Coding-Guide-for-Python/CWE-707/CWE-180/>
- OWASP Foundation. (2026). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>

- Thomson, M. (2018). The Harmful Consequences of the Robustness Principle. *IETF Internet-Draft*, draft-iab-protocol-maintenance.
- Internet Architecture Board. (2023). *RFC 9413: Maintaining Robust Protocols*. <https://www.rfc-editor.org/info/rfc9413/>
- Postel, J. (1980). *RFC 761: DoD Standard Transmission Control Protocol*.
- Carpenter, B. (Ed.) (1996). *RFC 1958: Architectural Principles of the Internet*. IAB.
- CVE-2026-42462. Fedify: JSON-LD imza doğrulamasında işlem sırası hatası. (CVSS 7.0.)